

Data Structures in Java

Lecture 7 & 8: Stack and Queue Applications

09/28/2016 &
10/3/2016

Daniel Bauer



Detecting Palindromes

- A palindrome is a word (or phrase) that contains the same sequence of letters, whether you read it forward or backward.
- *anna, bob, civic, kayak*
- “*a nut for a jar of tuna*”, “*i prefer pi*”, “*he won snow, eh*”

Balancing Symbols

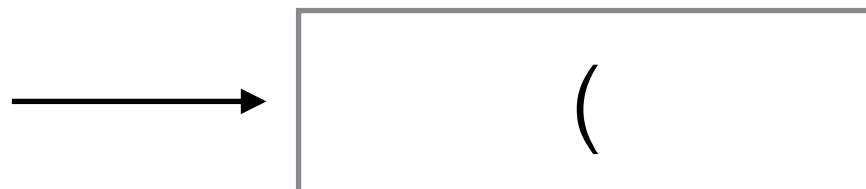
- Compilers need to check for syntax errors.
- Need to make sure braces, brackets, parentheses are well nested.
- What's wrong with this code:

```
for(int i=0;i<=topOfStack;i++) {  
    sb.append(theArray[i] + " ");  
sb.append("]");
```

Balancing Symbols

```
for(int i=0; i<=topOfStack; i++) {  
    sb.append(theArray[i] + " ");  
    sb.append("]");  
}
```

push("(")



Balancing Symbols

```
for(int i=0;i<=topOfStack;i++) {  
    sb.append(theArray[i] + " ");  
sb.append("]");
```

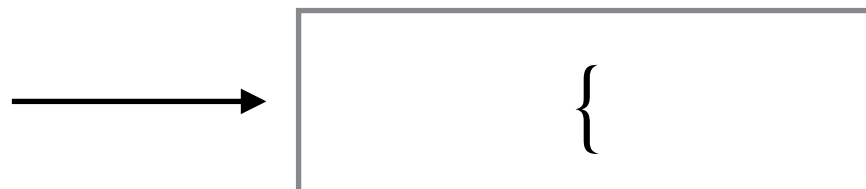
push("(") pop("(")



Balancing Symbols

```
for(int i=0;i<=topOfStack;i++) {  
    sb.append(theArray[i] + " ");  
sb.append("]");
```

push("(") pop("(") push("{")

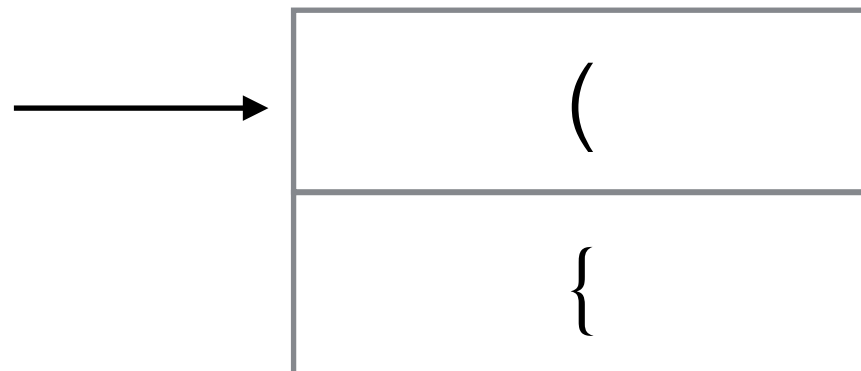


Balancing Symbols

```
for(int i=0;i<=topOfStack;i++) {  
    sb.append(theArray[i] + " ");  
sb.append("]");
```

push("(") pop("(") push("{")

push("(")

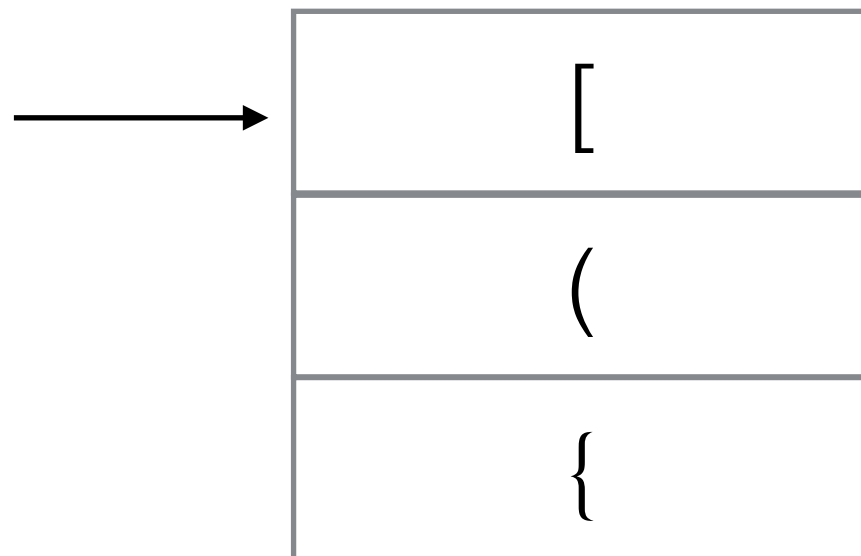


Balancing Symbols

```
for(int i=0;i<=topOfStack;i++) {  
    sb.append(theArray[i] + " ");  
sb.append("]");  
}
```

push("(") pop("(") push("{")

push("(") push("[")

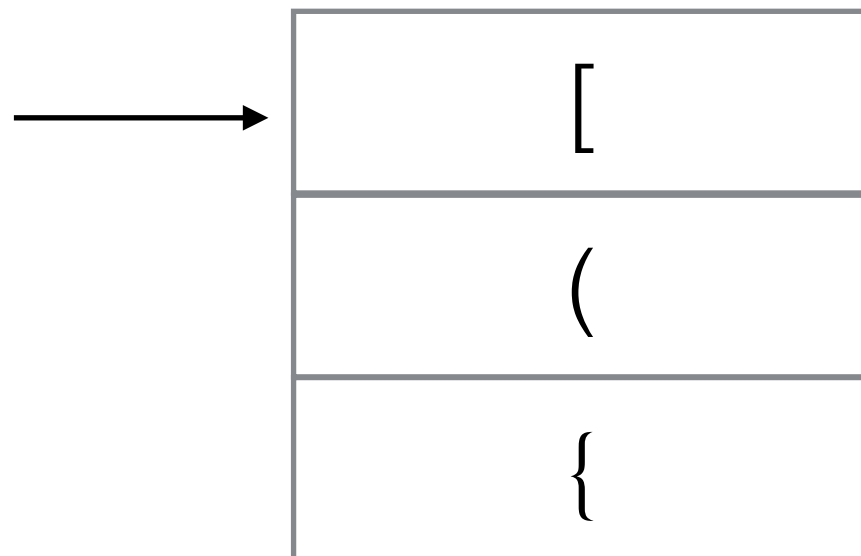


Balancing Symbols

```
for(int i=0;i<=topOfStack;i++) {  
    sb.append(theArray[i] + " ");  
sb.append("]");
```

push("(") pop("(") push("{")

push("(") push("[")



Postfix Expressions

- How would you do the following calculation using a **simple** calculator:

$$5 + 27 / (2 * 3)$$

remember
intermediate
results



Postfix Expressions

- How would you do the following calculation using a **simple** calculator:

$$5 + 27 / (2 * 3)$$

$$2 * 3 = 6$$

remember
intermediate
results



Postfix Expressions

- How would you do the following calculation using a **simple** calculator:

$$5 + 27 / (2 * 3)$$

$$2 * 3 = 6$$

$$27 / 6 = 4.5$$

remember
intermediate
results



Postfix Expressions

- How would you do the following calculation using a **simple** calculator:

$$5 + 27 / (2 * 3)$$

$$2 * 3 = 6$$

$$27 / 6 = 4.5$$

$$5 + 4.5 = 9.5$$

remember
intermediate
results



Postfix Expressions

- How would you do the following calculation using a **simple** calculator:

remember
intermediate
results

$$5 + 27 / (2 * 3)$$

$$2 * 3 = 6$$

$$27 / 6 = 4.5$$

$$5 + 4.5 = 9.5$$

5 27 2 3 * / +



Postfix Expressions

Postfix Expressions

5 27 2 3 * / +

Postfix Expressions

Postfix: 5 27 2 3 * / +

Postfix Expressions

Postfix: 5 27 2 3 * / +

5 + 27 / (2 * 3)

Postfix Expressions

Postfix: 5 27 2 3 * / +

Infix: 5 + 27 / (2 * 3)

Postfix Expressions

Postfix: 5 27 2 3 * / +

Infix: 5 + 27 / (2 * 3)

Prefix:

Postfix Expressions

Postfix: 5 27 2 3 * / +

Infix: 5 + 27 / (2 * 3)

Prefix: + 5 / 27 * 2 3

Evaluating Postfix Expressions

$$5 + 27 / (2 * 3)$$

5 27 2 3 * / +

- for c in input
 - if c is an operand, push it
 - if c is an operator x:
 - pop the top 2 operands a_1 and a_2
 - push $a_3 = a_2 \times a_1$
- pop the result.

Evaluating Postfix Expressions

$$5 + 27 / (2 * 3)$$

push(5)

5 27 2 3 * / +



- for c in input
 - if c is an operand, push it
 - if c is an operator x:
 - pop the top 2 operands a_1 and a_2
 - push $a_3 = a_2 \times a_1$
- pop the result.

Evaluating Postfix Expressions

$$5 + 27 / (2 * 3)$$

push(27)

5 27 2 3 * / +



- for c in input
 - if c is an operand, push it
 - if c is an operator x:
 - pop the top 2 operands a_1 and a_2
 - push $a_3 = a_2 \times a_1$
- pop the result.

Evaluating Postfix Expressions

$$5 + 27 / (2 * 3)$$

5 27 2 3 * / +

push(2)



2
27
5

- for c in input
 - if c is an operand, push it
 - if c is an operator x:
 - pop the top 2 operands a_1 and a_2
 - push $a_3 = a_2 \times a_1$
- pop the result.

Evaluating Postfix Expressions

$$5 + 27 / (2 * 3)$$

5 27 2 3 * / +

push(3)



3
2
27
5

- for c in input
 - if c is an operand, push it
 - if c is an operator x:
 - pop the top 2 operands a_1 and a_2
 - push $a_3 = a_2 \times a_1$
- pop the result.

Evaluating Postfix Expressions

$$5 + 27 / (2 * 3)$$

pop() -> 3
pop() -> 2
push(2*3)

5 27 2 3 * / +



6
27
5

- for c in input
 - if c is an operand, push it
 - if c is an operator x:
 - pop the top 2 operands a_1 and a_2
 - push $a_3 = a_2 \times a_1$
- pop the result.

Evaluating Postfix Expressions

$$5 + 27 / (2 * 3)$$

pop() -> 6
pop() -> 27
push(27/6)

5 27 2 3 * / +



- for c in input
 - if c is an operand, push it
 - if c is an operator x:
 - pop the top 2 operands a_1 and a_2
 - push $a_3 = a_2 \times a_1$
- pop the result.

Evaluating Postfix Expressions

$$5 + 27 / (2 * 3)$$

pop() -> 4.5
pop() -> 5
push(5 + 4.5)

5 27 2 3 * / +

→ 9

- for c in input
 - if c is an operand, push it
 - if c is an operator x:
 - pop the top 2 operands a_1 and a_2
 - push $a_3 = a_2 \times a_1$
- pop the result.

Converting Infix to Postfix Notation

Input : $a + b * c + (d * e + f) * g$

Output :

Converting Infix to Postfix Notation

Input : $a + b * c + (d * e + f) * g$

Output : $a b c * + d e * f + g * +$

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output:

Order of Precedence:

$+ = 1$

$* = 2$

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: a + b * c + d

Output: a

Order of Precedence:

+ = 1

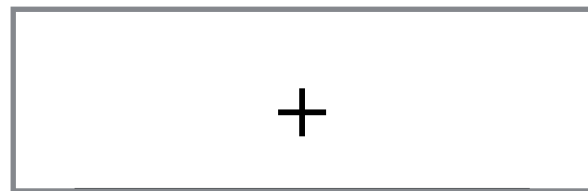
* = 2

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: a



Order of Precedence:

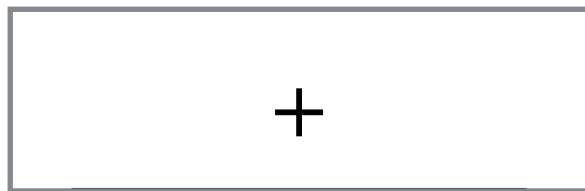
$+ = 1$
 $* = 2$

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + \boxed{b}^* c + d$

Output: $a \ b$



Order of Precedence:

$+ = 1$

$* = 2$

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: $a \ b$

*
+

Order of Precedence:

$+ = 1$

$* = 2$

$*$ has higher priority than $+$,
so we want $*$ in the output first. Keep pushing.

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: $a \ b \ c$

*
+

Order of Precedence:

$+ = 1$

$* = 2$

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: $a \ b \ c$

*
+

Order of Precedence:

$+ = 1$

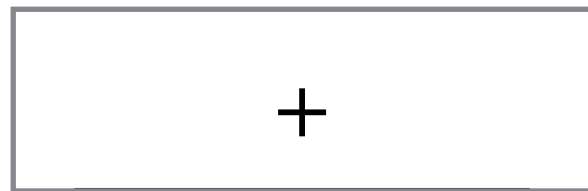
$* = 2$

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: $a \ b \ c \ *$



Order of Precedence:

$+ = 1$
 $* = 2$

$+$ has lower priority than $*$, so we need to pop $*$ and write it to the output first.

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: $a \ b \ c \ * \ +$

Order of Precedence:

$+ = 1$

$* = 2$

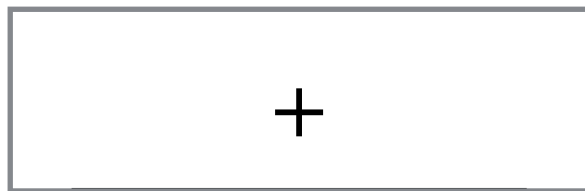
Need to pop the first $+$ too to keep sequential order.

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: $a \ b \ c \ * \ +$



Order of Precedence:

$+ = 1$
 $* = 2$

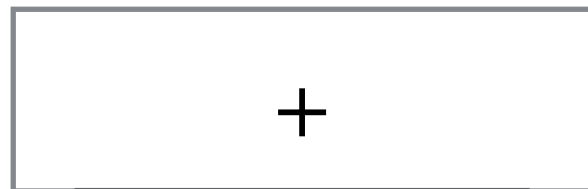
Then push the new +

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: $a \ b \ c \ * \ + \ d$



Order of Precedence:

$+ = 1$
 $* = 2$

Then push the new +

Converting Infix to Postfix Notation

Idea: keep lower-precedence operators on the stack.

Input: $a + b * c + d$

Output: $a \ b \ c \ * \ + \ d \ +$

Order of Precedence:

$+ = 1$

$* = 2$

Pop remaining stack elements.

Converting Infix to Postfix Algorithm Sketch

- for c in input
 - if c is an operand: print c
 - if c is "+", "*":
 - while stack is not empty and $\text{priority}(\text{stack.top}()) \geq \text{priority}(c)$:
 - print stack.pop()
 - push c
- while stack is not empty:
print stack.pop()

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a



Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a

(
*

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: $a * (b + c) * d + e$

Output: $a b$

(
*

Order of Precedence:

$+ = 1$

$* = 2$

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: $a * (b + c) * d + e$

Output: $a b$

+
(
*

Order of Precedence:

$+ = 1$

$* = 2$

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: $a * (b + c) * d + e$

Output: $a b c$

+
(
*

Order of Precedence:

$+ = 1$

$* = 2$

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c

+
(
*

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c +

(
*

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c +

*

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c +

*

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c + *

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: $a * (b + c)$ * $d + e$

Output: $a \ b \ c \ + \ *$

*

Order of Precedence:

$+ = 1$

$* = 2$

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: $a * (b + c) * \boxed{d} + e$

Output: $a \ b \ c \ + \ * \ d$

$*$

Order of Precedence:

$+ = 1$

$* = 2$

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c + * d

*

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c + * d *

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c + * d *

+

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c + * d * e

+

Order of Precedence:

+ = 1

* = 2

Converting Infix to Postfix

Dealing with ()

Idea: Put “(“ on stack. When “)” is seen, reduce stack until matching “(“.

Input: a * (b + c) * d + e

Output: a b c + * d * e +

Order of Precedence:

+ = 1

* = 2

Stacks in Hardware

- Stack as a memory abstraction:
 - CPU implement a hardware stack (use register to point to “top” location in main memory).
 - CPU operations push, pop will write/get value and increase or decrease register with a single byte code instruction.

Stack Machines

- Most modern computers are register machines.
To compute $2+3$:
 - `mov eax, 2`
 - `move ebx, 3`
 - `add eax, ebx` which stores the result in `eax`
- In a Stack Machine:
 - `push 2`
 - `push 3`
 - `add` which stores the result back on the stack.
- Hardware stack machines are rare, but most virtual machines (including JVM) are stack machines.

What's wrong with this program?

```
public class Factorial {  
  
    public static int factorial(int n) {  
        return factorial(n-1) * n;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(factorial(10));  
    }  
}
```

What's wrong with this program?

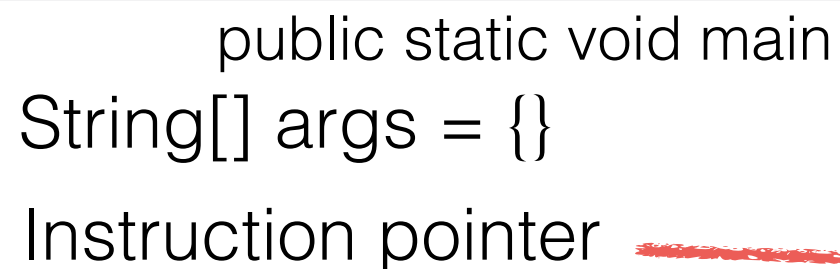
```
public class Factorial {  
  
    public static int factorial(int n) {  
        return factorial(n-1) * n;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(factorial(10));  
    }  
}
```

```
$ javac Factorial.java  
$ java Factorial  
Exception in thread "main" java.lang.StackOverflowError  
    at InfiniteRecursion.factorial(Factorial.java:4)  
    at InfiniteRecursion.factorial(Factorial.java:4)  
    at InfiniteRecursion.factorial(Factorial.java:4)
```

...

Method Call Stacks

- Every function keeps an *activation record* on the method call stack.
- Represent current state of execution of this function.
- Includes instruction pointer, value of variables, parameters, intermediate results.



```
public static void main
String[] args = {}
Instruction pointer
```

```
public static int factorial(int n) {
    return factorial(n-1) * n;
}

public static void main(String[] args) {
    System.out.println(factorial(10));
}
```

Method Call Stacks (2)

- When a function is called
 - Execution of the current function is suspended.
 - A new activation record is pushed to the stack.
 - The new function is run.

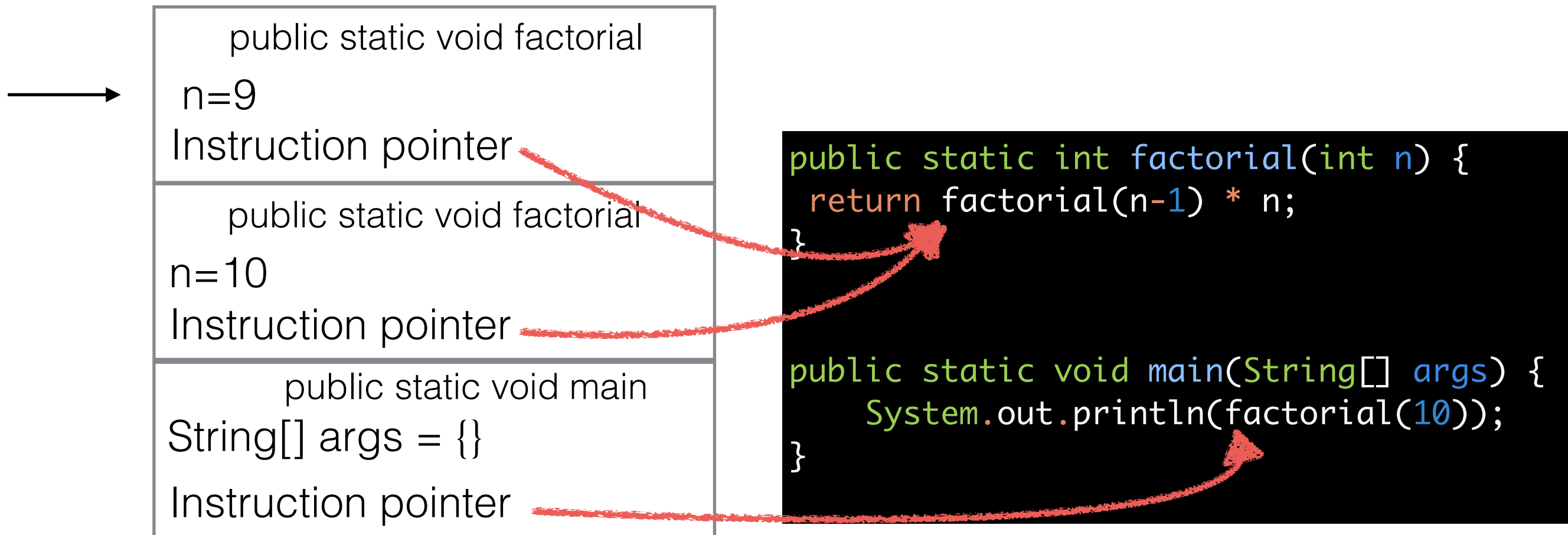
→
public static void factorial
n=10
Instruction pointer

public static void main
String[] args = {}
Instruction pointer

```
public static int factorial(int n) {  
    return factorial(n-1) * n;  
}  
  
public static void main(String[] args) {  
    System.out.println(factorial(10));  
}
```

Runaway Recursion

- Recursion will quickly grow the method call stack.
- Execution of the current function is suspended.



Fixing Runaway Recursion

- We forgot to add the base case:

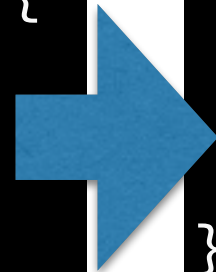
```
public static int factorial(int n) {  
    if (i == 1)  
        return 1;  
    return factorial(n-1) * n;  
}
```

- Still can get stack overflows for large n .

Rewriting Recursion

- This is a stupid use for recursion.

```
public static int factorial(int n) {  
    if (i == 1)  
        return 1;  
    return factorial(n-1) * n;  
}
```



```
public static int factorial(int n) {  
    int result = 1;  
    for (i = 1; i <= n; i++)  
        result = result * i;  
}
```

- In general, any recursion can be removed.
- Recursion is often more readable.

Tail Recursion

- Some compilers can detect and remove some types of recursion (the Java Compiler does not).
- A method is *tail recursive* if the last thing it does before returning is call itself. Compilers can turn this into a loop.

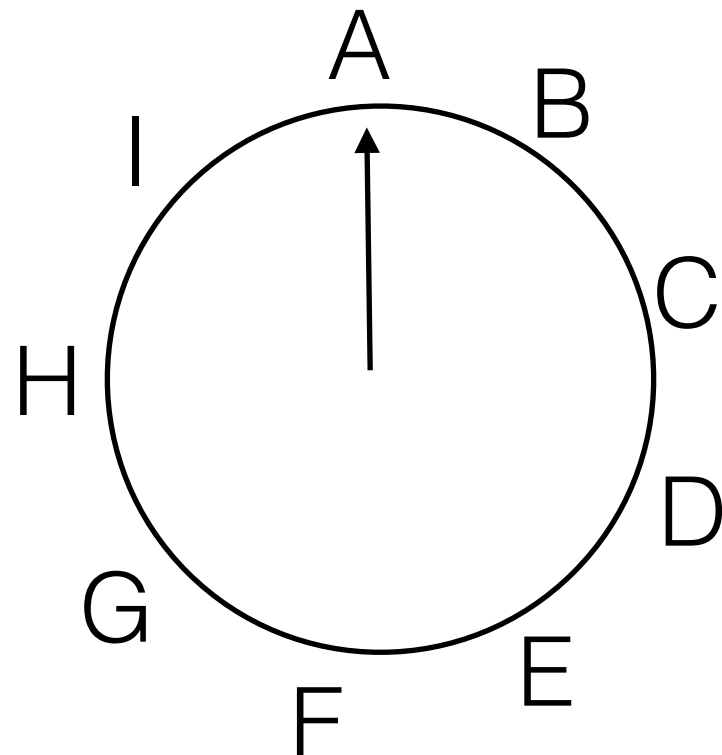
```
public static long factorial(long n) {  
    return facRec(n, 1);  
}  
  
public static long facRec(long n, long result) {  
    if (n==1)  
        return result;  
    else  
        return facRec(n-1, result * n);  
}
```


Counting Out Game with a Queue

- n players stand in a circle. The players are counted, proceeding in clockwise direction.
- The first $k-1$ players are skipped. The k -th player is “counted out”. Then the next $k-1$ players are skipped etc.
- The player who remains is the winner.

$n=9$

$k=4$

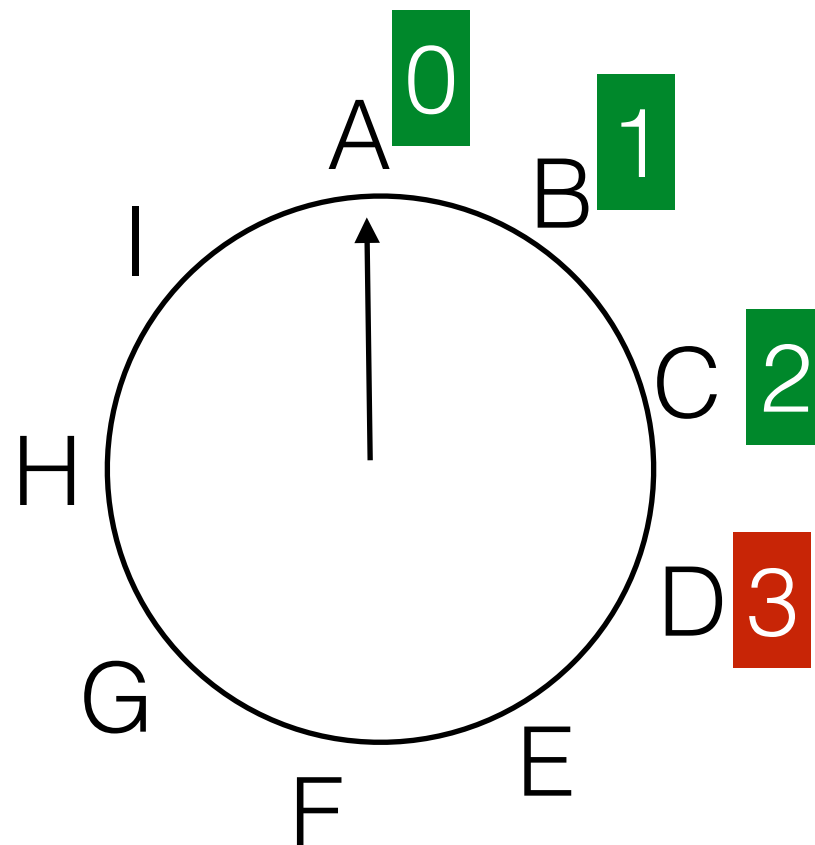


Counting Out Game with a Queue

- n players stand in a circle. The players are counted, proceeding in clockwise direction.
- The first $k-1$ players are skipped. The k -th player is “counted out”. Then the next $k-1$ players are skipped etc.
- The player who remains is the winner.

$n=9$

$k=4$

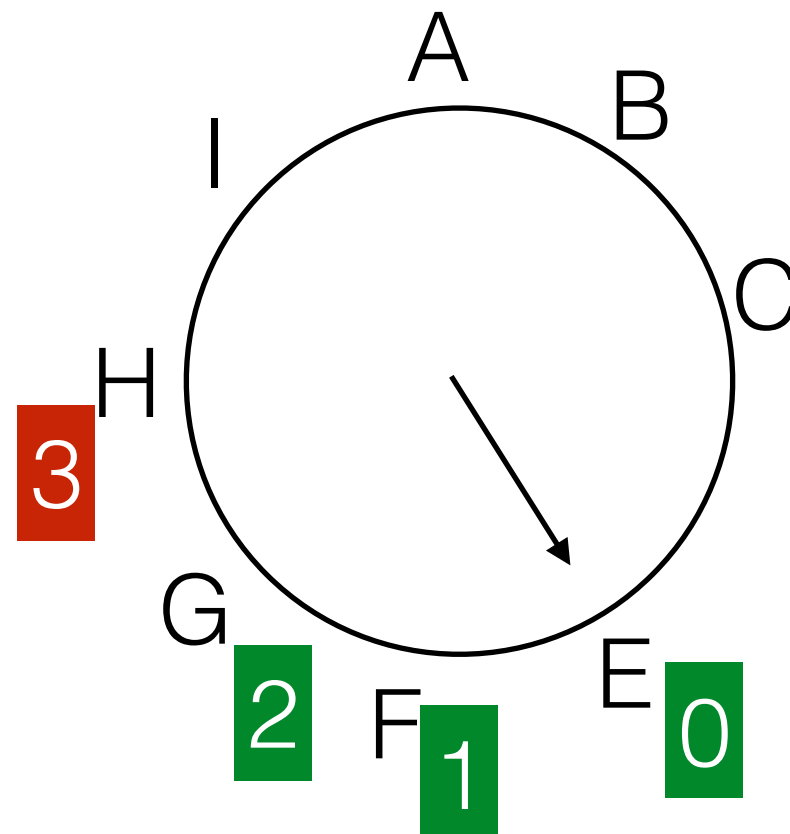


Counting Out Game with a Queue

- n players stand in a circle. The players are counted, proceeding in clockwise direction.
- The first $k-1$ players are skipped. The k -th player is “counted out”. Then the next $k-1$ players are skipped etc.
- The player who remains is the winner.

$n=9$

$k=4$

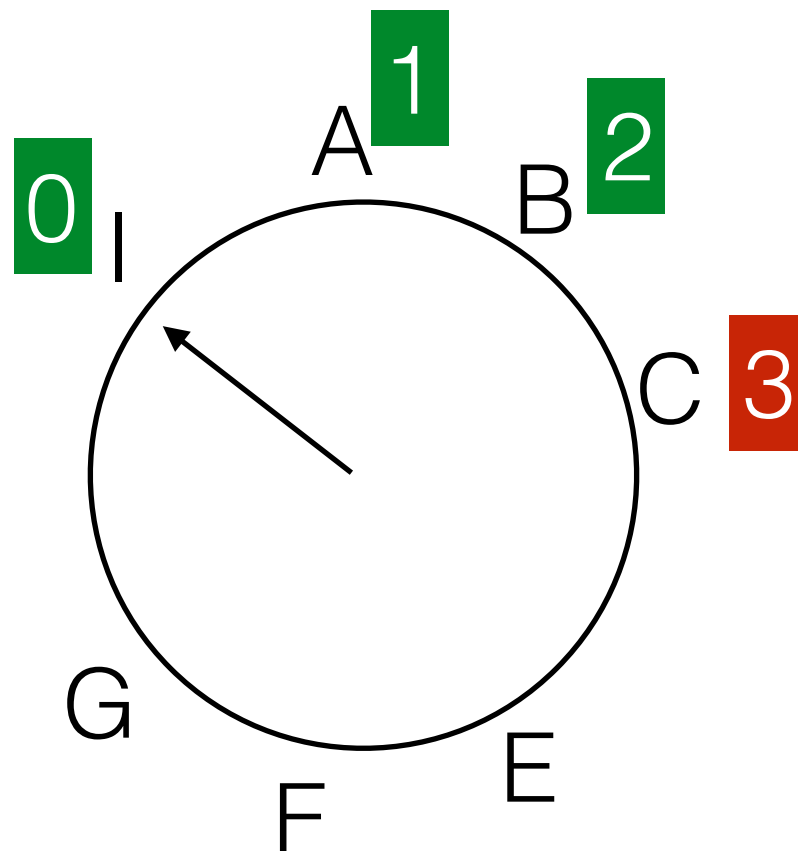


Counting Out Game with a Queue

- n players stand in a circle. The players are counted, proceeding in clockwise direction.
- The first $k-1$ players are skipped. The k -th player is “counted out”. Then the next $k-1$ players are skipped etc.
- The player who remains is the winner.

$n=9$

$k=4$

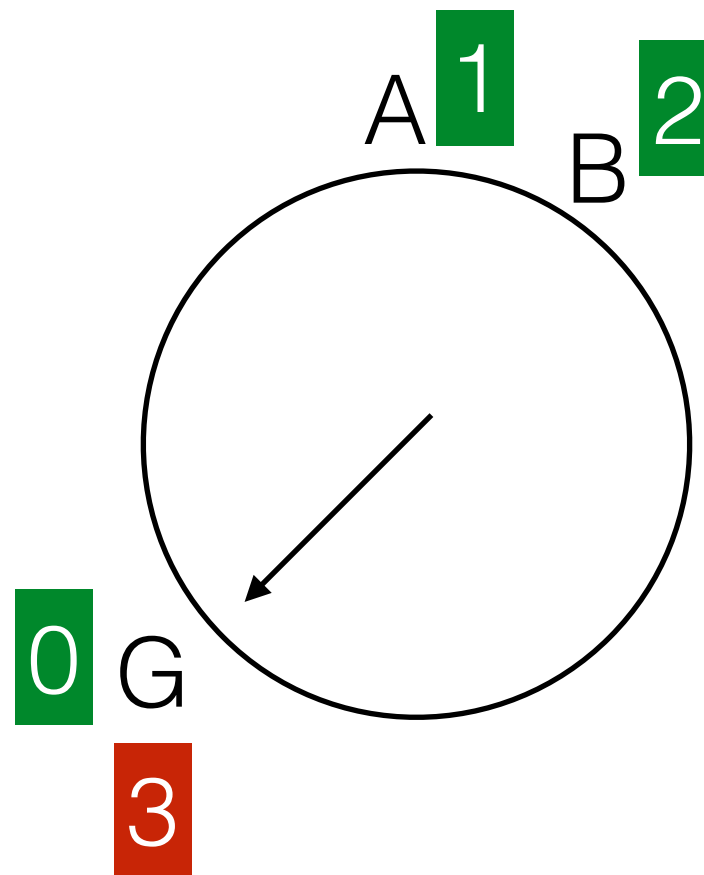


Counting Out Game with a Queue

- n players stand in a circle. The players are counted, proceeding in clockwise direction.
- The first $k-1$ players are skipped. The k -th player is “counted out”. Then the next $k-1$ players are skipped etc.
- The player who remains is the winner.

$n=9$

$k=4$

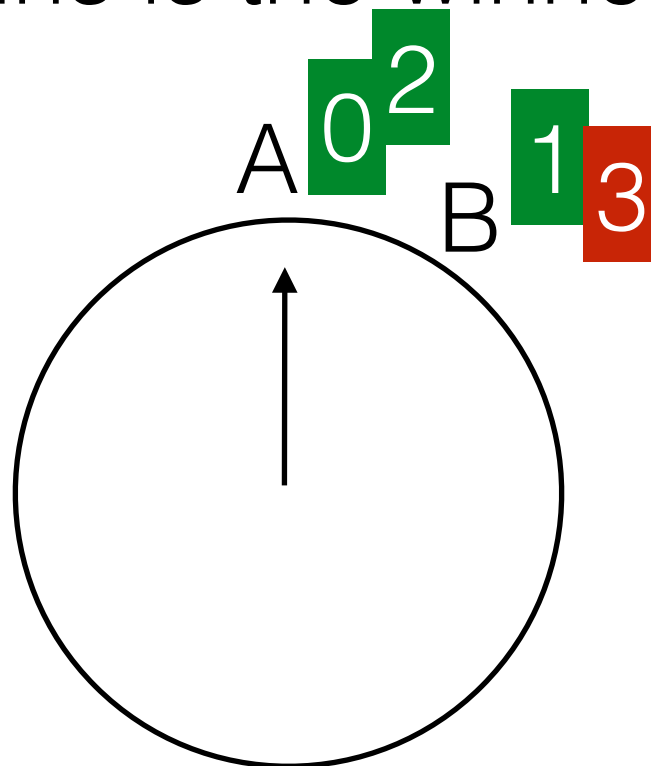


Counting Out Game with a Queue

- n players stand in a circle. The players are counted, proceeding in clockwise direction.
- The first $k-1$ players are skipped. The k -th player is “counted out”. Then the next $k-1$ players are skipped etc.
- The player who remains is the winner.

$n=9$

$k=4$



Counting Out Game with a Queue

- n players stand in a circle. The players are counted, proceeding in clockwise direction.
- The first $k-1$ players are skipped. The k -th player is “counted out”. Then the next $k-1$ players are skipped etc.
- The player who remains is the winner.

$n=9$

$k=4$

